CENG-567
REINFORCEMENT LEARNING

Lecture-5

Markov Chains, Markov Reward Process, Markov Decision Process

Dr. Osman GÖKALP

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

Contextual Bandits (CBs)

What are they? An advanced Multi-Armed Bandit where the action a is chosen based on an observed context x.

Goal: Learn a policy to maximize rewards by estimating the action value, $Q(a|x)$.

Key Distinction from Full RL: In CBs, an action leads to a reward but does not influence the next context. It is a single-step decision problem.

Success Metric: Minimizing Cumulative Regret—the difference in reward between our chosen action and the optimal action over time.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

The Challenge of Dynamic Actions & The DNN Solution

The Problem: In real-world systems like ad networks, the set of available actions (e.g., ads) is massive and constantly changing. Creating a separate model for each action is impossible.

The Solution: Use a single, unified model that takes both context features (x) and action features (a) as input to estimate the value $Q(x, a)$.

Function Approximator: For complex relationships, Deep Neural Networks (DNNs) are the natural choice to learn the function $Q(x, a)$.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

Thompson Sampling with DNNs: The Intractability Problem

Thompson Sampling (TS): A powerful exploration strategy that samples from the posterior distribution of model parameters.

The Problem: For complex models like DNNs, calculating the true posterior is computationally **intractable**.

The Solution: Approximate Bayesian Inference

We use **Monte Carlo (MC) Dropout**.

The Trick: Keep dropout layers **active during inference (prediction)**. Each forward pass produces a different output, simulating a sample from the model's posterior distribution.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Markov Property

Markov property: the characteristic of a process in which the **next state** depends only on the **current state**, not on any past states.

This concept is named after the Russian mathematician **Andrey Markov**.

What is a "State"?

Before understanding the property, we must define a "state.“

A state is the set of information that completely and sufficiently **describes the situation an environment** is in.

Think of it as a "**snapshot**" of the world.

If the environment is in the same state at two different times, it means everything about the environment is exactly the same.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example: A broken robot that randomly moves in a grid world

- The robot is currently in state (1,2). It does not matter where it has come from; it will be in state (1,3) with a probability of 0.2, in (0,2) with a probability of 0.25 , and so on.
- Since the probability of where it will **transition** next **depends only** on which state it is **currently** in, but not where it was before, the process has the **Markov property**.

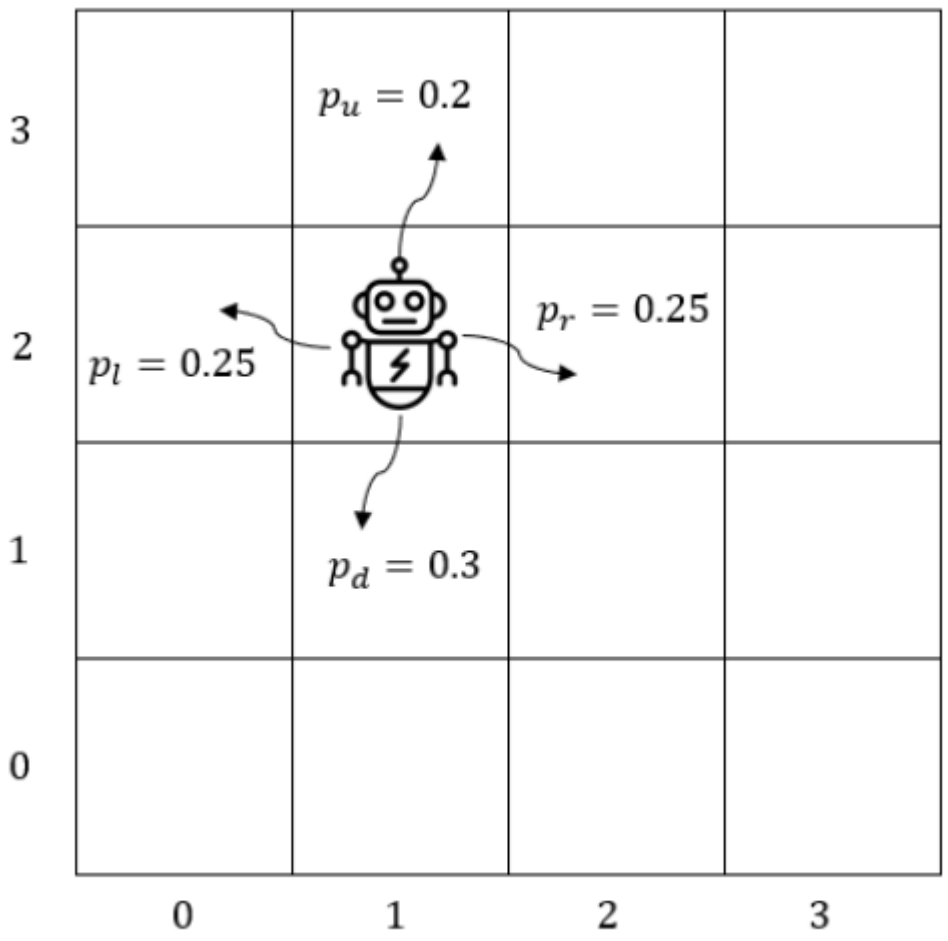


Figure 4.1 – A broken robot in a grid world, currently at (1,2)

Markov Property and Markov Chain

Notation:
Let s_t denote the **state** at time t.

Markov Property:
A process satisfies the **Markov property** if the next state depends **only on the current state**, not on previous ones:

$$P(\boldsymbol{s_{t+1}} \mid s_t, s_{t-1}, s_{t-2}, \dots, s_0) = P(\boldsymbol{s_{t+1}} \mid s_t)$$

Markov Chain:
A **stochastic process** that satisfies the Markov property is called a **Markov chain**.

Example:

Assume that if a robot **hits a wall**, it **bounces back** and **remains in the same state**.

For instance, while in state **(0,0)**, the robot stays there in the next step with probability **0.55** $(\mathrm{p_l} + \mathrm{p_d})$.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Markov Chain Diagram for the Robot Example (using **2x2** grid world)

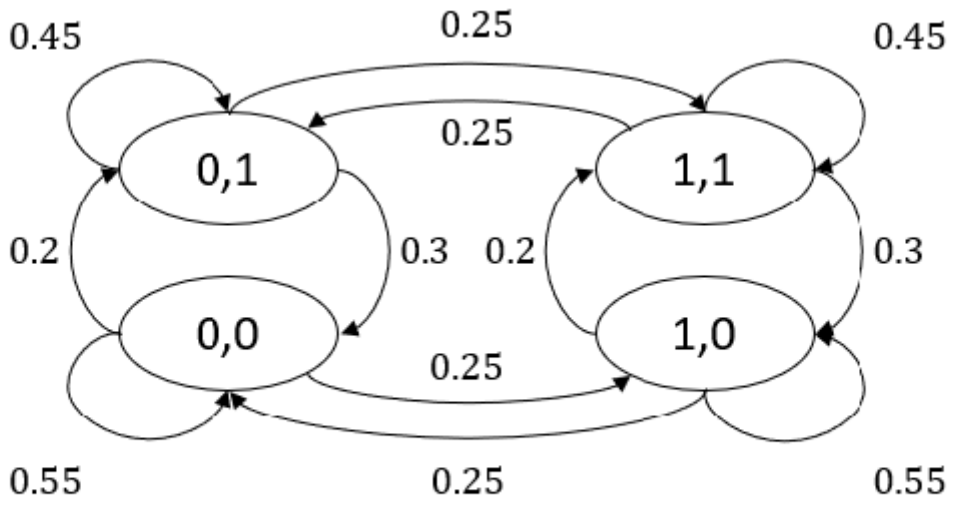


Figure 4.2 – A Markov chain diagram for the robot example in a 2x2 grid world

Note

Many systems can be made **Markovian** by including historical information in the state.

Consider a modified robot example where the robot is more likely to continue in the direction it moved in the previous time step.

Although such a system seemingly does not satisfy the **Markov property**, we can simply redefine the state to include the visited cells over the last two time steps, such as

$$s_t = (x_t, x_{t-1}) = \big((0,1),(0,0)\big)$$

Under this new state definition, the transition probabilities become independent of earlier states, and the **Markov property** is satisfied.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Classification of States in a Markov chain

Reachable and Communicating States

If the environment can transition from **state** s to **state** s' after some number of steps with a **positive probability**, then s' is *reachable* from s.

If s is also reachable from s', the two states are said to **communicate**.

If **all states** in a Markov chain communicate with each other, the chain is called **irreducible**.

In our robot example, the Markov chain is **irreducible** since all states communicate.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Classification of States in a Markov chain

Absorbing States

A **state** s is an **absorbing state** if the only possible transition is **to itself**:

$$P(s_{t+1} = s \mid s_t = s) = 1$$

Example:
If the robot crashes into a wall and can no longer move, that state becomes **absorbing**, since the robot can never leave it.

In a **2×2 grid world**, such an absorbing state can be shown in a **Markov chain diagram** (see the *figure right*).

In **Reinforcement Learning (RL)**, an **absorbing state** is equivalent to a **terminal state**, marking the **end of an episode**.
Episodes can also end when a **time limit** T is reached.

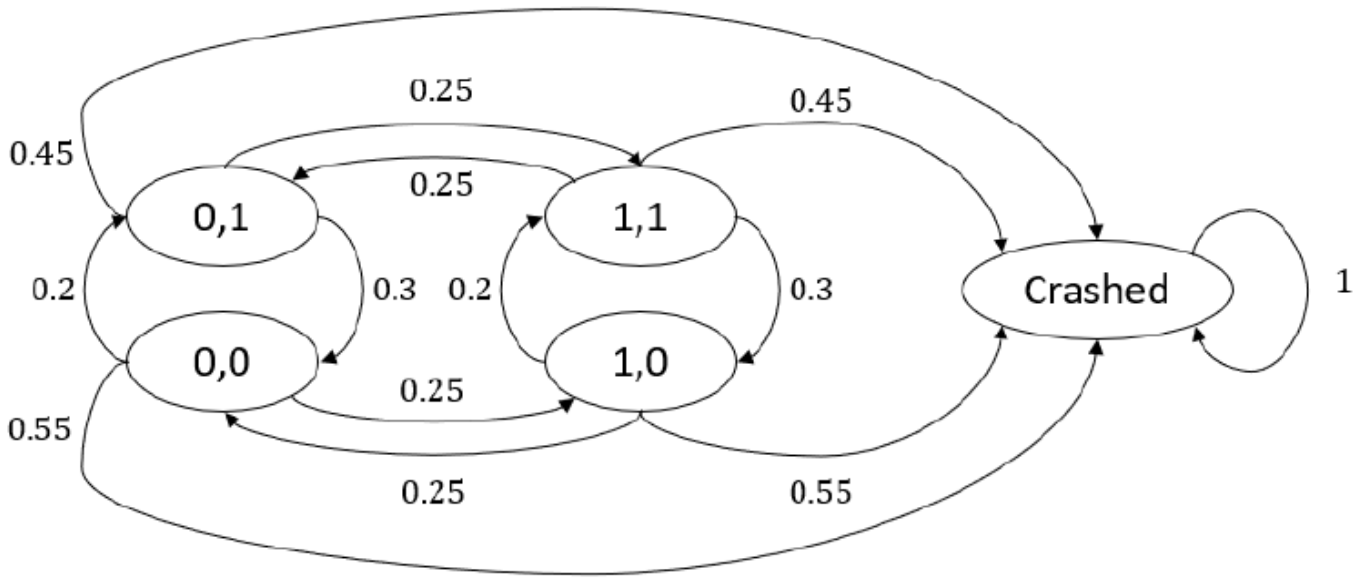


Figure 4.3 – A Markov chain diagram with an absorbing state

Classification of States in a Markov chain

Transient States

A **state** s is called a **transient state** if there exists another state s' that is **reachable from** s, but s **is not reachable from** s'.

Over time, the environment will eventually **leave transient states** and **never return**.

A **state** that is **not transient** is called a **recurrent state**.

** Identify which state(s) are transient and which are recurrent?*

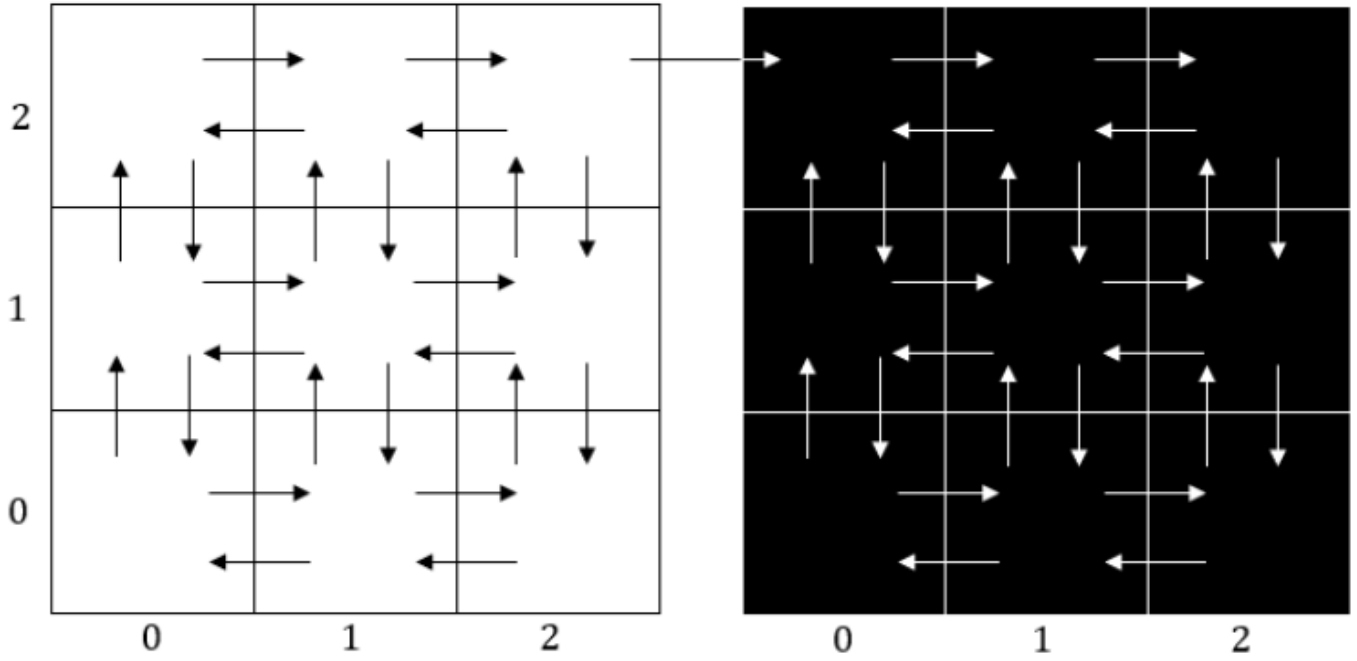


Figure 4.4 – Grid world with a light and dark side

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Classification of States in a Markov chain

Periodic and Aperiodic States

A **state** s is **periodic** if all paths leaving s return after some multiple of $d > 1$steps (d = GCD (greatest common divisor)).

Example: In *figure right*, **all states** have a **period** $d = 4$.

A **recurrent state** is called **aperiodic** if

$$d = 1$$

Key idea: Aperiodic states can be revisited at irregular intervals, while periodic states can only be revisited at multiples of their period.

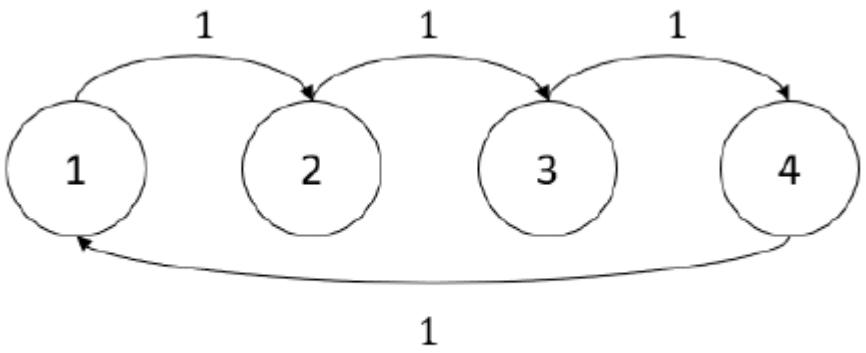


Figure 4.5 – A Markov chain with periodic states, k=4

Note: self-transition guarantees aperiodicity, but it is not necessary

(A state can have return times like 2,3,5,6, ...(no 1-step return) with GCD = 1 → still aperiodic)

Classification of States in a Markov chain

Ergodicity

A **Markov chain** is called **ergodic** if all states satisfy:

- **Communicate with each other** (irreducible)
- **Are recurrent**
- **Are aperiodic**

Key idea:

In an **ergodic Markov chain**, the system eventually "forgets" its initial state.

Steady-State Probability

For **ergodic chains**, we can calculate a **steady-state probability distribution**:

Describes the **long-term probability** of being in each state.

This distribution is **independent of the initial state**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Transitionary and Steady State Behavior

Modeling Markov Chains Mathematically

To calculate how a **Markov chain behaves over time**, we need:

- **Initial probability distribution** q
  - Example: In a grid world, which state the robot starts in.
- **Transition probability matrix** P
  - Entry at row i, column j:

$$P_{ij} = P(s_{t+1} = j \mid s_t = i)$$

  - Gives the probability of transitioning from **state** i to **state** j in one step.

Probability After Multiple Steps

To calculate the probability of being in **state** i after n steps: $\mathrm{p_n} = q\, P^{\mathrm{n}}$

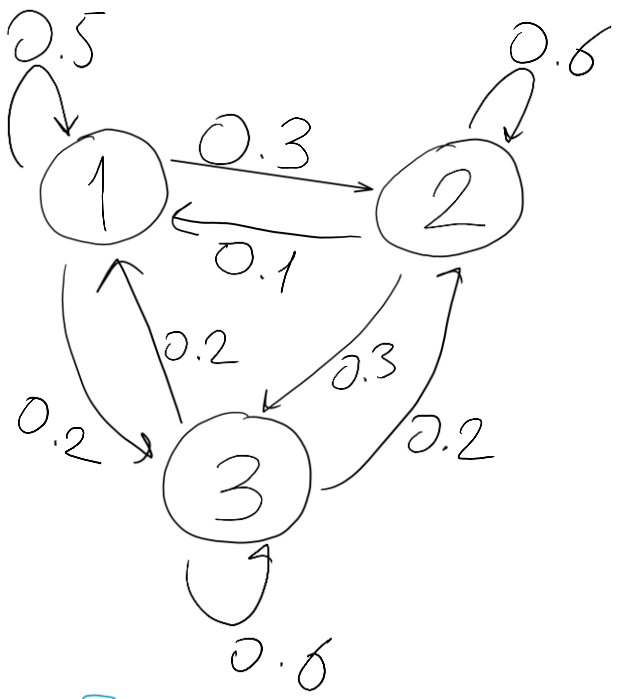
q =initial probability distribution

P^{n} =transition matrix raised to the n-th power

$P_{ij}{}^{\mathrm{n}}$ gives the probability of being in state after steps when started in state .

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example: 3-state Markov Chain, n=2 steps



$$P = \begin{bmatrix} 0.5 & 0.3 & 0.2 \\ 0.1 & 0.6 & 0.3 \\ 0.2 & 0.2 & 0.6 \end{bmatrix}$$

$$P^2 = P \times P = \begin{bmatrix} 0.32 & 0.37 & 0.31 \\ 0.17 & 0.45 & 0.38 \\ 0.24 & 0.3 & 0.46 \end{bmatrix}$$

$$P_r(X_2 = 3 \mid X_0 = 1) = P^2_{1,3} = 0.31$$

Let $q = [0.2, 0.5, 0.3]$

$$q \times P^2 = [0.221, 0.389, 0.390]$$

$P_r(X_2 = 2) = 0.389$ under initial q.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: n-step behavior in the grid world

See '**MarkovDecisionProcess.ipynb**' file in the Teams course page.

Many RL algorithms aim to achieve **consistency** between:

- Our understanding of the environment in the **current state**
- Our predictions after **n steps of transitions**

Iteration continues until this **consistency is achieved**.

Understanding **how a Markov chain evolves over time** is crucial.

Next step: Analyze **n-step behavior** in the **grid world example**.

Implementation: n-step behavior in the grid world

Let's start by creating a 3 x 3 grid world (9 states) with our robot in it, similar to the one in *Figure 4.1*. For now, let's always initialize the world with the **robot** being **at the center**. Moreover, we index the states/cells so that (0,0): 1, (0,1): 2,..., (2,2): 9. So, the initial probability distribution, q, is given by the following code:

```
import numpy as np
m = 3
m2 = m ** 2
q = np.zeros(m2)
q[m2 // 2] = 1
```

q:

0	0	0	0	**1**	0	0	0	0

State indices order

(2,2)

7	8	9
4	5	6
1	2	3

(0,0)

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: n-step behavior in the grid world

- Define a function (`get_P`) that gives the $n \times n$ transition probability matrix.
- Then get the transition probability matrix for a given parameters.

p_up: 0.2
p_down: 0.3
P_left: 0.25
p_right: 0.25

```
P: [[0.55 0.25 0.   0.2  0.   0.   0.   0.   0.  ]
    [0.25 0.3  0.25 0.   0.2  0.   0.   0.   0.  ]
    [0.   0.25 0.55 0.   0.   0.2  0.   0.   0.  ]
    [0.3  0.   0.   0.25 0.25 0.   0.2  0.   0.  ]
    [0.   0.3  0.   0.25 0.   0.25 0.   0.2  0.  ]
    [0.   0.   0.3  0.   0.25 0.25 0.   0.   0.2 ]
    [0.   0.   0.   0.3  0.   0.   0.45 0.25 0.  ]
    [0.   0.   0.   0.   0.3  0.   0.25 0.2  0.25]
    [0.   0.   0.   0.   0.   0.3  0.   0.25 0.45]]
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: n-step behavior in the grid world

- Then calculate n-step transition probabilities (n=1,3,10,100)

```
n = 10
Pn = np.linalg.matrix_power(P, n)
res = np.matmul(q, Pn)
print(np.round(res, 3))
```

$n = 1$

0	0.2	0
0.25	0	0.25
0	0.3	0

$n = 3$

0.068	0.107	0.068
0.137	0.061	0.137
0.124	0.176	0.124

$n = 10$

0.072	0.072	0.072
0.105	0.106	0.105
0.156	0.156	0.156

$n = 100$

0.07	0.07	0.07
0.105	0.105	0.105
0.158	0.158	0.158

Figure 4.6 – n-step transition probabilities

You might notice that the probability distribution after 10 steps and 100 steps are very similar. This is because the system has almost reached a **steady state after a few steps**. So, the chance that we will find the robot in a specific state is almost the same after 10, 100, or 1,000 steps. Also, you should have noticed that we are more likely to find the robot at the **bottom cells**, simply because we have p_down > p_up .

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Estimating the Steady-State Distribution

If a **Markov chain is ergodic**, we can estimate its **steady-state distribution** by **simulation**.

Approach:

Run the chain for a **long time**.

Record the **frequency of visits** to each state.

These frequencies approximate the **steady-state probabilities**.

Why useful?

When **transition probabilities** are **unknown**,
but the system can be **simulated** directly.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Estimating the Steady-State Distribution

- Initialize the first state to 4, which is (1, 1)
- Simulate the environment for one million steps
- Count the number of visits to each state
- The results are indeed very much in line with the steady state probability distribution we calculated.

```
s = 4
n = 10 ** 6
visited = [s]

for t in range(n):
    s = np.random.choice(m2, p=P[s, :])
    visited.append(s)

unique, counts = np.unique(visited, return_counts=True)
print(np.asarray((unique+1, counts)).T)
```

```
[[     1 157340]
 [     2 158360]
 [     3 158757]
 [     4 104856]
 [     5 104976]
 [     6 105146]
 [     7  70440]
 [     8  70148]
 [     9  69978]]
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Semi-Markov Processes and Continuous-time Markov Chains

Beyond Discrete-Time Markov Chains

So far, we studied **discrete-time Markov chains**,
where transitions occur at **fixed intervals** (e.g., every minute).

In many real-world systems, **transition times are random** →
these are called **Semi-Markov Processes (SMPs)**.

In such systems, we are interested in the **state after a certain amount of time**,
not just after a certain number of steps.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Semi-Markov Processes and Continuous-time Markov Chains

Understanding Semi-Markov Processes

To describe a **semi-Markov process**, we need:

- The **current state**, and
- How long the system has been in that state.

The system depends on the **past in terms of time**,
but not in terms of **transition type** → hence *"semi"-Markov*.

Possible cases:

Ignore time: Use the **embedded Markov chain** → behaves like discrete-time.

Random but memoryless time intervals:

- Transition times are **exponentially distributed**.
- The system satisfies the **Markov property** →
  becomes a **Continuous-Time Markov Chain (CTMC)**.

Non-memoryless time + time matters:

- A **general semi-Markov process**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Semi-Markov Processes and Continuous-time Markov Chains

Relation to Reinforcement Learning (RL)

In RL, continuous-time environments are often **approximated as discrete**.

Common practice:

- Use **discrete-time RL algorithms** with suitable **workarounds**.

For now:

- Understand the **conceptual differences**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Semi-Markov Processes and Continuous-time Markov Chains

Aspect	Memoryless (Markov / CTMC)	Memory-Based (Semi-Markov / Non-Markov)
Definition	Next state depends **only on the current state**.	Next state depends on **current state and how long the system has been in it**, or even earlier states.
Mathematical property	$P(X_{t+1} = x' \mid X_t = x)$	$P(X_{t+1} = x' \mid X_t = x, X_{t-1} = x_{t-1}, \tau_t)$
Time between transitions	Fixed or **exponentially distributed** (memoryless waiting time).	**Arbitrary** (may depend on time spent in state).
Memory property (waiting time)	$(P(T > t + s \mid T > s) = P(T > t)$ → **no memory**	$P(T > t + s \mid T > s) \neq P(T > t)$ → **depends on history**
Example systems	Discrete grid world, Poisson processes, **continuous-time Markov chains (CTMC), embedded Markov chains**.	Queuing systems with variable service times, machine repair with aging, **semi-Markov decision processes (SMDPs)**.
Relevance in RL	Standard assumption in most RL algorithms (Markov Decision Process).	Requires extensions such as **semi-Markov Decision Processes (SMDPs)** or approximations for time-aware RL.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Markov Reward Process

Context:
So far, in our robot (grid world) example, all states were treated equally —
we did **not distinguish** between *good* or *bad* states.

Key Idea:
In real systems, some states are **more desirable** than others.
To capture this, we assign **rewards** to states or transitions.

Markov Reward Process (MRP):

Extends a Markov Chain by adding **rewards**

Each state (or transition) has an associated **reward value**

Allows us to compute the **value** of each state —
how "good" it is to be there in the long run.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Attaching rewards to the grid world example

- From now on, we will work on this version: when robot hits a wall it crashed in a way that it was **not recoverable**, it could not bounce back to the cell it was in.

1. We first modify the transition probability matrix to assign self-transition probabilities to the "crashed" state that we add to the matrix:

```
P = np.zeros((m2 + 1, m2 + 1))
P[:m2, :m2] = get_P(3, 0.2, 0.3, 0.25, 0.25)
for i in range (m2):
  P[i, m2] = P[i, i]
  P[i, i] = 0
P[m2, m2] = 1
```

```
[[0.   0.25 0.   0.2  0.   0.   0.   0.   0.   0.55]
 [0.25 0.   0.25 0.   0.2  0.   0.   0.   0.   0.3 ]
 [0.   0.25 0.   0.   0.   0.2  0.   0.   0.   0.55]
 [0.3  0.   0.   0.   0.25 0.   0.2  0.   0.   0.25]
 [0.   0.3  0.   0.25 0.   0.25 0.   0.2  0.   0.  ]
 [0.   0.   0.3  0.   0.25 0.   0.   0.   0.2  0.25]
 [0.   0.   0.   0.3  0.   0.   0.   0.25 0.   0.45]
 [0.   0.   0.   0.   0.3  0.   0.25 0.   0.25 0.2 ]
 [0.   0.   0.   0.   0.   0.3  0.   0.25 0.   0.45]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   1.  ]]
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Attaching rewards to the grid world example

2. For every transition where the robot **stays alive**, it collects **+1 reward**. It collects 0 reward when it crashes. Since **"crashed" is a terminal/absorbing state**, we terminate the episode there. **Simulate** this model for different initializations, **100K times for each initialization**, and see how much **reward is collected on average in each case**.

```
n = 10 ** 5
avg_rewards = np.zeros(m2)
for s in range(m2):
  for i in range(n):
    crashed = False
    s_next = s
    episode_reward = 0
    while not crashed:
      s_next= np.random.choice(m2 + 1, p=P[s_next, :])
      if s_next == m2:
        crashed = True
      else:
        episode_reward += 1
    avg_rewards[s] += episode_reward
  avg_rewards[s] /= n
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Attaching rewards to the grid world example

3. The results will look as in *Figure 4.7* (yours will be a bit different due to the randomness):

2	2.00	2.81	1.99
1	2.45	3.41	2.44
0	1.48	2.12	1.46
	0	1	2

Figure 4.7 – Average returns with respect to the initial state

In this example, if the initial state is (1, 1) , **the average return** is the **highest**. This makes it a **"valuable" state** to be in. Contrast this with state (0, 2) with an average return of 1.46 . Not surprisingly, it is not a great state to be in. This is because it is more likely for the robot to hit the wall earlier when it starts in the corner. Another thing that is not surprising is that the returns are **vertically symmetrical** (almost), since p_left = p_right.

Recursive Relationship Between State Values

Idea:
Average returns from states are structurally related.

If the robot starts at **(1,1)** and moves to **(1,2)**:

It receives a reward **+1** for staying alive.

If we already know the value of **state (1,2)**,
we don't need further simulation —
that value already gives the **expected return** from there onward.

Because this is a **Markov process**,
the **future** depends only on the **current state**, not the past.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recursive Relationship Between State Values

State Value Relationship:

$v(1,1)=p_{up}(1+v(2,1))+p_{down}(1+v(0,1))+p_{left}(1+v(1,0))+p_{right}(1+v(1,2))$

$$= 0.2(1 + 2.81) + 0.3(1 + 2.12) + 0.25(1 + 2.45) + 0.25(1 + 2.44)$$
$$= 3.42 \approx 3.41$$

As you can see, some small inaccuracies in the estimations of the state values aside, the state values are consistent with each other.

This **recursive relationship** between state values is at the heart of many **Reinforcement Learning algorithms**.

We will **formalize** this idea using the **Bellman equation** in the next section.

Return, Discount, and State Values

Return after time step t (R_T is the terminal):

$$G_t = R_{t+1} + R_{t+2} + \cdots + R_T$$

However, if there is **no terminal state**, the return can grow **infinitely**.

To avoid this, introduce a discount factor $0 \leq \gamma \leq 1$:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

At time t, the next reward you will see is R_{t+1}, after that the one following is R_{t+2}...

Effect of γ:

- $\gamma \approx 1$: future and immediate rewards weighted almost equally
- $\gamma = 1$: all rewards weighted equally
- $\gamma \approx 0$: short-sighted (myopic) behavior
- $\gamma = 0$: only the immediate reward counts

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Return, Discount, and State Values

Reasons for discounting:

Reduces weight on uncertain future rewards

Reflects **preference for immediate rewards** (human/animal behavior)

Matches **financial reasoning** (time value of money)

State Value (value of a state) Definition:

$$V(s) = \mathbb{E}[G_t \mid S_t = s] = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s\right]$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Bellman Equation for MRP

Recursive relationship for state values:

$$V(s) = \sum_{s',r} p(s', r \mid s)\ [r + \gamma V(s')]$$

This is called the **Bellman Equation** for a **Markov Reward Process (MRP)**.

Key Points:

Expresses the **value of a state** in terms of **its successor states**.

Reflects the **expected discounted return** over all possible transitions.

Forms the **foundation** of many Reinforcement Learning algorithms.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Formal Definition of an MRP

An **MRP** is fully defined by the tuple: $< \mathrm{S}, \mathrm{P}, \mathrm{R}, \gamma >$

where:

S: Set of **states**

P: **Transition probability** matrix

R: **Reward function**

$\gamma \in [0, 1]$:**Discount factor**

🧮 Analytically Calculating State Values

Key Idea:

The **Bellman equation** relates **state values**, **rewards**, and **transition probabilities**.

If **transition probabilities** and **rewards** are known, we can **compute state values precisely**.

Feasible only for **small state spaces** due to computational limits.

Matrix form of the Bellman equation:

$$\mathbf{v} = \mathbf{P}\mathbf{R} + \gamma \mathbf{P}\,\mathbf{v}$$

Where:

$\mathbf{v}$= column vector of **state values**

$\mathbf{R}$= column vector of **state rewards**

$\mathbf{P}$= **state transition probability matrix**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

🧮 Analytically Calculating State Values

Expanded Representation

$$\begin{bmatrix} v(1) \\ v(2) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathrm{P}_{1,1} & \dots \mathrm{P}_{1,\mathrm{n}} \\ & \vdots \\ \mathrm{P}_{\mathrm{n},1} & \dots \mathrm{P}_{\mathrm{n},\mathrm{n}} \end{bmatrix} \begin{bmatrix} R(1) \\ R(2) \\ \vdots \\ R(n) \end{bmatrix} + \gamma \begin{bmatrix} \mathrm{P}_{1,1} & \dots \mathrm{P}_{1,\mathrm{n}} \\ & \vdots \\ \mathrm{P}_{\mathrm{n},1} & \dots \mathrm{P}_{\mathrm{n},\mathrm{n}} \end{bmatrix} \begin{bmatrix} v(1) \\ v(2) \\ \vdots \\ v(n) \end{bmatrix}$$

Each entry $v(i)$ depends on the **expected reward** from transitioning into all possible next states plus the **discounted values** of those next states.

From the matrix form:

$$\mathbf{v} = \mathbf{PR} + \gamma \mathbf{Pv}$$

We can rearrange terms:

$$(\mathbf{I} - \gamma \mathbf{P})\mathbf{v} = \mathbf{PR}$$

Thus, the exact solution is:

$$\mathbf{v} = (\mathbf{I} - \gamma \mathbf{P})^{-1}\mathbf{PR}$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Analytically Calculating State Values

- Now let's implement this for our **grid world example**.
  In the **3 × 3 environment**, there are **10 states in total**, where the **10th state** corresponds to the robot **crashing**.
  Each transition into a state yields a **reward of +1**, **except** when the robot ends up in the **crashed state**.

```
R = np.ones(m2 + 1) # +1 is for the extra state crash
R[-1] = 0 # crash state (last state) has reward 0
```

```
gamma = 0.9999
inv = np.linalg.inv(np.eye(m2 + 1) - gamma * P) #np.eye is the identity matrix (I)
v = np.matmul(inv, np.matmul(P, R))
print(np.round(v, 2))
```

```
[1.47 2.12 1.47 2.44 3.42 2.44 1.99 2.82 1.99 0.  ]
```

Remember that these are the true (theoretical, rather than estimated) state values (for the given discount rate), and they are aligned with what we estimated through simulation before in *Figure 4.7*!

Implementation: Analytically Calculating State Values

If you're wondering why we didn't simply set **$\gamma = 1$**, remember that the **discount factor** ensures **mathematical convergence**.

In theory, the robot could keep moving indefinitely, collecting an **infinite reward**, even if it's extremely unlikely in practice.

However, setting **$\gamma = 1$** would make **$(I - \gamma P)$** a **singular matrix**, meaning it cannot be inverted. (P typically has eigenvalue 1; this prevents inversion and can lead to no solution or infinitely many solutions)

Therefore, we use **$\gamma = 0.9999$**, which still gives **nearly equal weight** to immediate and future rewards while keeping the system solvable.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Estimating the state values iteratively

One of the central ideas in **Reinforcement Learning (RL)** is to estimate the **value function** *iteratively* using its own definition as an update rule.

We **initialize** all state values arbitrarily.

Then, we **update** each state's value based on the estimated values of subsequent states.

Since we use **other estimates to update estimates**, this is called **bootstrapping**.

The process continues until the **maximum change** in state values across all states is **below a threshold**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Estimating the state values iteratively

```python
def estimate_state_values(P, m2, threshold):
  v = np.zeros(m2 + 1)
  max_change = threshold
  terminal_state = m2
  while max_change >= threshold:
    max_change = 0
    for s in range(m2 +1):
      v_new = 0
      for s_next in range(m2 + 1):
        r = 1 * (s_next != terminal_state)
        v_new += P[s, s_next] * (r + v[s_next])
      max_change = max(max_change, np.abs(v[s] - v_new))
      v[s] = v_new
  return np.round(v, 2)
```

```python
estimate_state_values(P, m2, 0.01)
```

```
array([1.46, 2.11, 1.47, 2.44, 3.41, 2.44, 1.98, 2.82, 1.99, 0.  ])
```

Limitations of Direct Value Iteration

Drawbacks:

Requires **iterating over all possible states**, which becomes **intractable** for large state spaces.

Relies on **explicit transition probabilities**, which are often **unknown** in real systems.

Modern Solution:
Reinforcement Learning (RL) algorithms overcome these issues by:

Using **function approximation** to represent state values.

Sampling transitions directly from the (simulated) environment instead of relying on full probability models.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Markov Decision Process (MDP)

From MRP to MDP:

In an **MRP**, we can model and study a **Markov chain with rewards**.

In an **MDP**, we add **decisions (actions)** that influence how the system evolves.

Definition

An **MDP** is simply an **MRP with actions** — where decisions affect both the **transition probabilities** and the **rewards**.

Formal Representation: MDP $= \langle S, A, P, R, \gamma \rangle$

S → Set of states, **A** → Set of actions, **P** → Transition probabilities, **R** → Reward function, **γ** → Discount factor

💡 **Key Insight:**
MDP is the **mathematical foundation of Reinforcement Learning (RL)** — it provides the structure for decision-making under uncertainty.

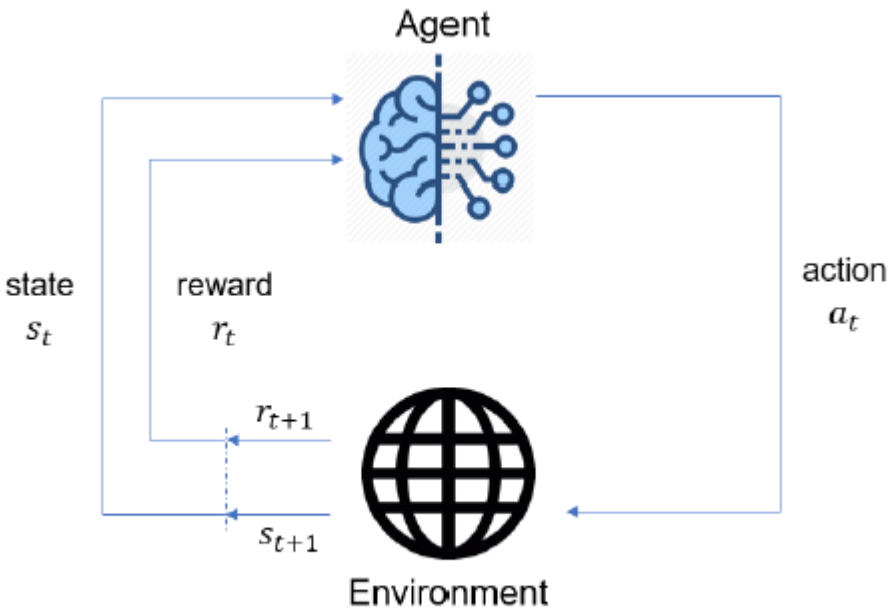


Figure 4.8 – MDP diagram

Markov Decision Process (MDP)

Goal:
Find a **policy** that **maximizes** the **expected cumulative reward**.

Policy (π):
A policy defines the agent's **behavior** — it specifies which action(s) to take in each state.

$$\pi(a \mid s) = P[A_t = a \mid S_t = s]$$

Maps **states → actions** (or distributions over actions)

Stationary: does not change over time

Determines both **transitions** and **rewards**

MDP Dynamics:
The environment's behavior under a policy is defined by the **transition probabilities**:

$$p(s', r \mid s, a) = P(S_{t+1} = s', R_t = r \mid S_t = s, A_t = a)$$

This holds for **all** states s and actions a.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Grid World as an MDP

We can now **control** the robot in the grid world.

Available **actions**: up, down, left, right.

The robot moves:

in the **intended direction** with probability **0.7**,
in one of the **other directions** with probability **0.1** each.

Example Policy: Right when in state (0, 0); **Up** when in state (1, 0)

Transition Probabilities (for state (0,0), action = right)

$$P(S_{t+1} = (1,0), R_t = 1 \mid S_t = (0,0), A_t = \text{right}) = 0.7$$
$$P(S_{t+1} = (0,1), R_t = 1 \mid S_t = (0,0), A_t = \text{right}) = 0.1$$
$$P(S_{t+1} = \text{crashed}, R_t = 0 \mid S_t = (0,0), A_t = \text{right}) = 0.2$$

🟩 **Tip:** Once a policy is defined in an MDP, the state–reward sequence becomes a **Markov Reward Process (MRP).**

Grid World as an MDP

Deterministic policy example:

- saying that "the policy is to take the "right" action in state (0, 0) " actually means "we take the "right" action with probability 1 when in state (0, 0) ."

$$\pi(\text{ right } \mid (0,0)) = 1$$

Stochastic policy example:

$$\pi(\text{ up } \mid (1,0)) = 0.5$$
$$\pi(\text{ left } \mid (1,0)) = 0.5$$

Goal in RL

Find the **optimal policy** that maximizes the **expected discounted return**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

State-Value Function under a Policy

The value of a state, or the **state-value function**, is the expected discounted return when starting in state $s$and following **policy $\boldsymbol{\pi}$**.

The **policy affects transition probabilities**, so each policy **leads to a different value function**.

$$v_\pi(s) = E_\pi[\, G_t \mid S_t = s \,] = E_\pi[\, \textstyle\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \,]$$

The subscript π indicates the policy being followed. The idea is the same as in MRP, except that transitions now depend on actions.

Bellman Equation for $v_\pi(s)$

The Bellman equation expresses $v_\pi(s)$ recursively in terms of next states, actions, and rewards:

$$v_\pi(s) = \sum_a \pi(\, a \mid s \,) \sum_{s',r} p(\, s', r \mid s, a \,)[r + \gamma\, v_\pi(s')]$$

We already know that the values of the states are related to each other from our discussion on MRPs. The only difference here is that now the transition probabilities depend on the actions and the corresponding probabilities of taking them in a given state as per the policy. Imagine "no action" is one of the possible actions in our grid world. The state values in Figure 4.7 would correspond to the policy of taking no action in any state.

Action-Value Function: Definition

The **action-value function**, $q_\pi(s, a)$, gives the expected discounted return starting from state s, taking action a, and then following policy π thereafter.

It answers:
"What is the expected cumulative reward if I take action a now, and follow π afterward?"

$$q_\pi(s, a) = E_\pi[\, G_t \mid S_t = s, A_t = a \,]$$
$$= E_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,|S_t = s, A_t = a]$$
$$= E[R_{t+1} + \gamma \mathrm{v}_\pi(\mathrm{s}')|S_t = s, A_t = a]$$
$$= \sum_{s',r} p(\, s', r \mid s, a \,)\, [\mathrm{r} + \gamma \mathrm{v}_\pi(\mathrm{s}')]$$

- We can improve our policy by choosing the action that gives the highest action value for state s, represented by argmax $\arg\max_{\mathrm{a}} q_\pi(s, a)$.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Optimal State-Value and Action-Value Functions

Optimal State-Value Function

An **optimal policy** is one that achieves the highest possible state value:

$$v_*(s) = \max_{\pi} v_\pi(s), \forall s \in S$$

π_*denotes an optimal policy. Multiple optimal policies may exist. However, **there is only one optimal state-value function** v_*.

Optimal Action-Value Function

$$q_*(s,a) = \max_{\pi} \mathrm{q}_\pi(s,\mathrm{a})$$

Relationship Between v_* **and** q_*

$$q_*(s,a) = \mathbb{E}\left[R_{t+1} + \gamma\, v_*(S_{t+1}) \mid S_t = s, A_t = a\right]$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Bellman optimality

When we defined the Bellman equation earlier for $v_\pi(s)$, we needed to use $\pi(\, a \mid s\,)$ in the equation. This is because the state-value function is defined for a policy and we needed to calculate the expected reward and the value of the following state with respect to the action(s) suggested by the policy while in state (together with the corresponding probabilities if multiple actions are suggested to be taken with positive probability). This equation was the following:

$$v_\pi(s) = \sum_a \pi(\, a \mid s\,) \sum_{s',r} p(\, s', r \mid s, a\,)[r + \gamma\, v_\pi(s')]$$

However, while dealing with the optimal state-value function, v_* , we don't really need to retrieve $\pi(\, a \mid s\,)$ from somewhere to plug into the equation.

Why? Because the optimal policy should be suggesting an action that maximizes the consequent expression.

After all, the state-value function represents the cumulative expected reward. If the optimal policy was not suggesting the action that maximizes the expectation term, it would not be an optimal policy.

Therefore, for the optimal policy and the state-value function, we can write a special form of the Bellman equation. This is called the **Bellman optimality equation.**

Bellman optimality

Bellman optimality equation (<u>state-value</u>):

$$\mathrm{v}_*((\mathrm{s}))=\max_a \mathrm{q}_*((\mathrm{s},a))$$
$$= \max_a E[R_{t+1} + \gamma v_*(s')|S_t = s, A_t = t]$$
$$= \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma\, v_*(s')]$$

We can write the **Bellman optimality equation for the <u>action-value</u>** function similarly:

$$q_\pi(s, a) = E[R_{t+1} + \gamma \mathrm{v}_*(\mathrm{s}')|S_t = s, A_t = a]$$
$$= E[R_{t+1} + \gamma \max_{a'} \mathrm{q}_*(S_{t+1}, a') \mid S_t = s, A_t = a]$$
$$= \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma \max_{a'} \mathrm{q}_*(s', a')\right]$$

- The Bellman optimality equation is one of the most central ideas in RL, which will form the basis of many of the algorithms we will introduce in upcoming lectures.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Partially observable MDPs (POMDP)

In standard MDPs, a policy maps **states** to actions:

$$\pi: S \rightarrow A$$

This assumes the agent always knows the full true state S_t.

State includes everything in an environment **related** to the agent's decision-making (in the grid world example, the **color of the walls** is not important, for instance, so it **would not be part of the state**).

But in many **real environments** (driving, robotics), the agent cannot observe everything — only **partial** and noisy **observations**.

Example:
A driver can only see what the windshield view allows — the full world state is hidden.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Partially observable MDPs (POMDP)

A POMDP extends MDPs to partially observable environments.

$$\text{POMDP} = \langle S, A, O, P, R, Z, \gamma \rangle$$

Where:

S: set of true states (not fully observable)

A: actions

O: set of possible observations

P: state transition probability

R: reward function

Z: observation function

γ: discount factor

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Partially observable MDPs (POMDP)

The agent does **not** observe the state s' directly.
Instead, after taking action a, it receives observation o:

$$Z(o, s', a) = P(O_{t+1} = o \mid S_{t+1} = s', A_t = a)$$

The observation function describes how likely the agent is to perceive a given observation when the environment transitions to s'.

Because the agent does not know the true state, a policy must depend on **observation history**, not just the current observation.

$$\pi(o_{1:t}) \rightarrow a_t$$

Single observations are insufficient (e.g., one camera frame does not reveal object speed).

Using Recurrent Neural Networks (LSTM)

Memory of past observations is maintained using recurrent networks.

$$h_t = LSTM(o_t, h_{t-1})$$
$$a_t = \pi(h_t)$$

LSTMs enable the agent to **infer hidden state information** from observation sequences.

In practice, POMDP-based RL algorithms often use RNN/LSTM architectures to remember past observations and estimate the hidden state.